



Security Assessment



Item 3 & 4 Internal Report

June 2026

Prepared for ether.fi



Disclaimer

This document is intended for **internal** use only.

PR	Initial Commit Hash	Final Commit Hash
#132	93ff2389	f907a44c76
#136	17c51113a	17c51113a

M-01 Module does not check if token is withdrawable

Severity: Medium	Impact: Medium	Likelihood: Medium
Files: DeploySafeAssetRecoveryModule.s.sol	Status: Fixed	

Description: Safes can hold any token, but can withdraw only a particular set of whitelisted assets. All such assets are also subject to delayed withdrawals, which is enforced through the `_areAssetsWithdrawable` check inside `requestWithdraw()`

JavaScript

```
function _requestWithdrawal(address safe, address[] memory tokens, uint256[] memory amounts,
address recipient) internal {
    .....

    _areAssetsWithdrawable($, tokens);
    _cancelOldWithdrawal(safe);
    .....
}
```



Problem is that the module only checks if the token is a borrow or collateral token, but not if it is also withdrawable, which could allow the Safe owner to skip the withdraw delay and immediately take out withdrawable funds out of the contract (assuming they are not collateral or borrow tokens):

JavaScript

```
if (debtManager.isCollateralToken(token) || debtManager.isBorrowToken(token)) {  
    revert OnlySupportedTokensCannotBeRecovered();  
}
```

In addition it would also affect pending CashModule withdrawals or in-flight bridge operations.

Recommendations: In addition to the `isCollateralToken` & `isBorrowToken` checks, ensure the tokens are not withdrawable before allowing the transfer

Customer's response: Fixed in commit [3e9971c7](#)

Fix Review: Fixed

Low Severity Issues

L-01 Recovery modules permanently trap tokens with non-standard transfer behaviors due to strict full-balance sweeps and zero-balance postconditions

Severity: **Low**

Impact: **Low**

Likelihood: **Low**

Files:

[DeploySafeAssetRecoveryModule.s.sol](#)

Status: Fixed

Description: The recovery modules mandate that unsupported tokens be swept completely out of the contract in a single transaction. `SafeAssetRecoveryModule.recover` computes the exact total balance and enforces a strict post-transfer balance check if `(IERC20(token).balanceOf(safe) != 0) revert;`

For positively rebasing tokens (e.g., stETH) or interest-bearing tokens with rounding mechanics, there might be some dust leftovers due to the share based accounting in `transfer()` for such tokens that causes the transaction to revert.

Recommendations: Instead of the strict `== 0` postcondition, consider checking that the balances have moved to ensure transfer succeeded

Customer's response: Fixed in commit [c3e070fa](#)

Fix Review: Fixed - "The new `if (remaining >= amount)` check generally solves the issue, one small edge case to note here is that in case of ultra small transfers (e.g 2-3 wei) it might be possible that the rounded dust is `>=` then what was transferred, but such type of transfers are economically not justified in the first place "

Informational Issues

I-01. Add deployed code verification after deploying SafeAssetRecoveryModule

Description: For extra safety, after [deploying](#) the [SafeAssetRecoveryModule](#) through the [DeploySafeAssetRecoveryModule](#) script, consider checking that the deployed bytecode is exactly as expected to prevent a front-runner from injecting a malicious deployment.

Recommendations: Use the [VerifyBytecode.s.sol](#) script utility to validate that the CREATE3 deployment is correct.

Customer's response: Fixed in commit [e7b6ecc3b](#)

Fix Review: Fixed – *“The inline code check addresses the main concern here. However we still recommend to also run the Bytecode verification procedure as an additional safety step after the deployment”*

I-02. SafeAssetRecoveryModule cannot recover native ETH stuck in the Safe

Description: [EtherFiSafe](#) declares a [receive\(\)](#) [external payable {}](#) function, meaning it can accumulate native ETH from module interactions or even direct user deposits.

However, [SafeAssetRecoveryModule.recover\(\)](#) is ERC20-only: it calls [IERC20\(token\).balanceOf\(safe\)](#) to determine the amount and encodes an [IERC20.transfe\(recipient, amount\)](#) call executed through [execTransactionFromModule](#). Neither of these operations applies to native ETH.

As a result, native ETH that accumulates in the safe as dust or from unexpected sources has no recovery path through this module, leaving it permanently stuck unless a safe module (e.g., [EtherFiStakeModule](#), [AaveV3Module._supply\(ETH\)](#)) happens to consume it intentionally.

Recommendations: Consider if ETH should be recoverable and handle it in the module if necessary

Customer's response: Acknowledged

Fix Review: Acknowledged

I-03. SafeAssetRecoveryModule.recover() reverts for unhealthy Safes

Description: Every call to `execTransactionFromModule` triggers `EtherFiHook.postOpHook()`, which calls `debtManager.ensureHealth(safe)` for all modules except `CashModule`. This means `recover()` reverts whenever the safe is unhealthy, even though the token being recovered is not a collateral or borrow token and has no effect on the safe's debt position.

Recommendations: This issue is meant to serve as an observation over the behaviour of the module. Generally if the account is unhealthy it makes sense to restrict any asset moving operations until it regains health status. Although it currently seems safe, excluding the module from the postOpHook health check seems like unnecessary expansion of the risk surface, given how critical health checks are to the system.

Consider acknowledging this issue and just documenting it.

Customer's response: Acknowledged – *"This is intended behavior, to ensure that safe solvency remains a priority."*

Fix Review: Acknowledged

I-04. Recovery signatures lack deadlines allowing relayers to sweep arbitrary future deposits

Description: Recovery module signatures bind only the safe, token, and recipient alongside the module's isolated nonce. They lack deadlines. The nonce used is a per-module nonce in `ModuleBase` rather than the Safe's primary nonce, meaning users cannot cancel pending signatures via `EtherFiSafe.cancelNonce()`.

Because signatures do not expire and rely on an untrusted relayer for submission, a malicious or compromised relayer can indefinitely withhold a signature and execute it at an arbitrary future time. Furthermore, since `recover()` sweeps the entire token balance upon execution, any subsequent deposits of that same token can be unexpectedly drained to the original recipient.

Recommendation: Consider adding a deadline parameter to ensure signatures cannot be stashed for future unexpected use



Customer's response: Fixed in commit [f907a44c7](#)

Fix Review: Fixed



Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline. It is helpful for smart contract developers and security researchers during auditing and bug bounties.

Certora also provides services such as auditing, formal verification projects, and incident response.